[ ]:
```
Build a simple assistant by chaining multiple prompts or simulating dialogue.
```

[1]:
```
# ==========================================================================
# CELL 1: Setup
# ==========================================================================

print("AI Assistant (Prompt Chaining) Started")
```

```
AI Assistant (Prompt Chaining) Started
```

[2]:
```
# ==========================================================================
# CELL 2: Define Persona & Prompts
# ==========================================================================

persona_prompt = """
You are a friendly AI tutor.
Explain concepts in simple language.
Use examples.
Ask one follow-up question after each response.
"""

def generate_explanation(topic):
    return f"""
Machine learning is a part of AI where computers learn from data instead of being programmed.
Example: Netflix recommends movies based on your watch history.

Question: Can you think of another example related to {topic}?
"""

print("Persona and prompts ready")
```

```
Persona and prompts ready
```

[3]:
```python
# ==============================================================================
# CELL 3: First Interaction
# ==============================================================================

user_input = "Machine Learning"

print("User:", user_input)

response1 = generate_explanation(user_input)

print("\nAI Tutor:", response1)
```

```
User: Machine Learning

AI Tutor:
Machine learning is a part of AI where computers learn from data instead of being programmed.
Example: Netflix recommends movies based on your watch history.

Question: Can you think of another example related to Machine Learning?
```

[4]:
```python
# ==============================================================================
# CELL 4: User Reply
# ==============================================================================

user_reply = "YouTube recommendations"

print("User:", user_reply)
```

```
User: YouTube recommendations
```

[5]:
```python
# ==============================================================================
# CELL 5: Follow-up Response
# ==============================================================================

def follow_up(reply):
```

jupyter Untitled5 Last Checkpoint: 5 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[5]:
```python
# ============================================================================
# CELL 5: Follow-up Response
# ============================================================================

def follow_up(reply):
    return f"""
Yes, {reply} is a great example!
The system learns your preferences and suggests content.

Another example is spam email filtering.

Question: Do you know how spam detection works?
"""

response2 = follow_up(user_reply)

print("\nAI Tutor:", response2)
```

```
AI Tutor:
Yes, YouTube recommendations is a great example!
The system learns your preferences and suggests content.

Another example is spam email filtering.

Question: Do you know how spam detection works?
```

[6]:
```python
# ============================================================================
# CELL 6: Second User Reply
# ============================================================================

user_reply2 = "No"

print("User:", user_reply2)
```

jupyter Untitled5 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[6]:
```python
# ==============================================================================
# CELL 6: Second User Reply
# ==============================================================================

user_reply2 = "No"

print("User:", user_reply2)
```

```
User: No
```

[7]:
```python
# ==============================================================================
# CELL 7: Adaptive Response
# ==============================================================================

def adaptive_response(reply):
    if reply.lower() == "no":
        return """
No problem! Spam filters look for patterns like suspicious words and sender behavior.

Example: Emails with phrases like 'free money' are often marked as spam.

Question: Would you like a step-by-step explanation?
"""
    else:
        return "Great! Let's explore more."

response3 = adaptive_response(user_reply2)

print("\nAI Tutor:", response3)
```

```
AI Tutor:
No problem! Spam filters look for patterns like suspicious words and sender behavior.

Example: Emails with phrases like 'free money' are often marked as spam.
```

jupyter Untitled5 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
AI Tutor:
No problem! Spam filters look for patterns like suspicious words and sender behavior.

Example: Emails with phrases like 'free money' are often marked as spam.

Question: Would you like a step-by-step explanation?
```

[8]:

```python
# ============================================================================
# CELL 8: System Flow
# ============================================================================

print("SYSTEM FLOW:\n")

steps = [
    "1. User input",
    "2. Apply persona",
    "3. Generate explanation",
    "4. Ask follow-up",
    "5. User reply",
    "6. Generate follow-up response",
    "7. Adapt based on reply",
    "8. Continue conversation"
]

for step in steps:
    print(step)
```

```
SYSTEM FLOW:

1. User input
2. Apply persona
3. Generate explanation
4. Ask follow-up
5. User reply
6. Generate follow-up response
```

jupyter Untitled5 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```
2. Apply persona
3. Generate explanation
4. Ask follow-up
5. User reply
6. Generate follow-up response
7. Adapt based on reply
8. Continue conversation
```

[9]:

```python
# ============================================================================
# CELL 9: Conclusion
# ============================================================================

print("CONCLUSION:\n")

print("Prompt chaining creates interactive AI assistants.")
print("Persona ensures consistent tone.")
print("Follow-up questions improve engagement.")
print("Adaptive responses make conversation dynamic.")
```

```
CONCLUSION:

Prompt chaining creates interactive AI assistants.
Persona ensures consistent tone.
Follow-up questions improve engagement.
Adaptive responses make conversation dynamic.
```

[10]:

```python
# ============================================================================
# CELL 10: Improvements & Reflection
# ============================================================================

print("IMPROVEMENTS & REFLECTION:\n")

print("Initial Issues:")
print("- Responses were too long and unclear")
print("- No interaction with the user")
print("- Assistant did not adapt to user replies\n")
```

jupyter Untitled5 Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[10]:
```python
# ============================================================================
# CELL 10: Improvements & Reflection
# ============================================================================

print("IMPROVEMENTS & REFLECTION:\n")

print("Initial Issues:")
print("- Responses were too long and unclear")
print("- No interaction with the user")
print("- Assistant did not adapt to user replies\n")

print("Improvements Made:")
print("1. Added persona (friendly AI tutor)")
print("2. Limited responses to simple explanations")
print("3. Added follow-up questions after each response")
print("4. Used prompt chaining for multi-step interaction")
print("5. Added adaptive responses based on user input\n")

print("Final Result:")
print("- Consistent and clear responses")
print("- Interactive conversation flow")
print("- Better user engagement")
print("- Works like a simple real-world assistant\n")

print("Learning Outcome:")
print("- Prompt design controls AI behavior")
print("- Chaining prompts improves interaction")
print("- Persona helps maintain consistent tone")
```

```
IMPROVEMENTS & REFLECTION:

Initial Issues:
- Responses were too long and unclear
- No interaction with the user
- Assistant did not adapt to user replies
```

jupyter Untitled5 Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

```python
print("- Better user engagement")
print("- Works like a simple real-world assistant\n")

print("Learning Outcome:")
print("- Prompt design controls AI behavior")
print("- Chaining prompts improves interaction")
print("- Persona helps maintain consistent tone")
```

```
IMPROVEMENTS & REFLECTION:

Initial Issues:
- Responses were too long and unclear
- No interaction with the user
- Assistant did not adapt to user replies

Improvements Made:
1. Added persona (friendly AI tutor)
2. Limited responses to simple explanations
3. Added follow-up questions after each response
4. Used prompt chaining for multi-step interaction
5. Added adaptive responses based on user input

Final Result:
- Consistent and clear responses
- Interactive conversation flow
- Better user engagement
- Works like a simple real-world assistant

Learning Outcome:
- Prompt design controls AI behavior
- Chaining prompts improves interaction
- Persona helps maintain consistent tone
```

[ ]: